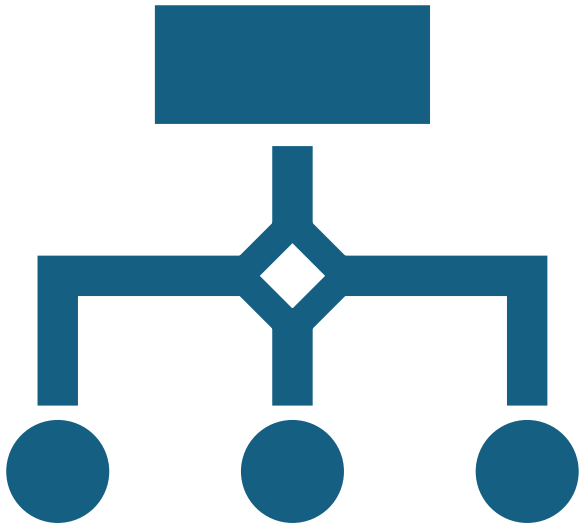
Introdução ao Git: um guia prático para iniciantes

Erick Sharlls Ramos de Pontes
erick.sharlls@ifpb.edu.br
https://github.com/ericksharlls

Git

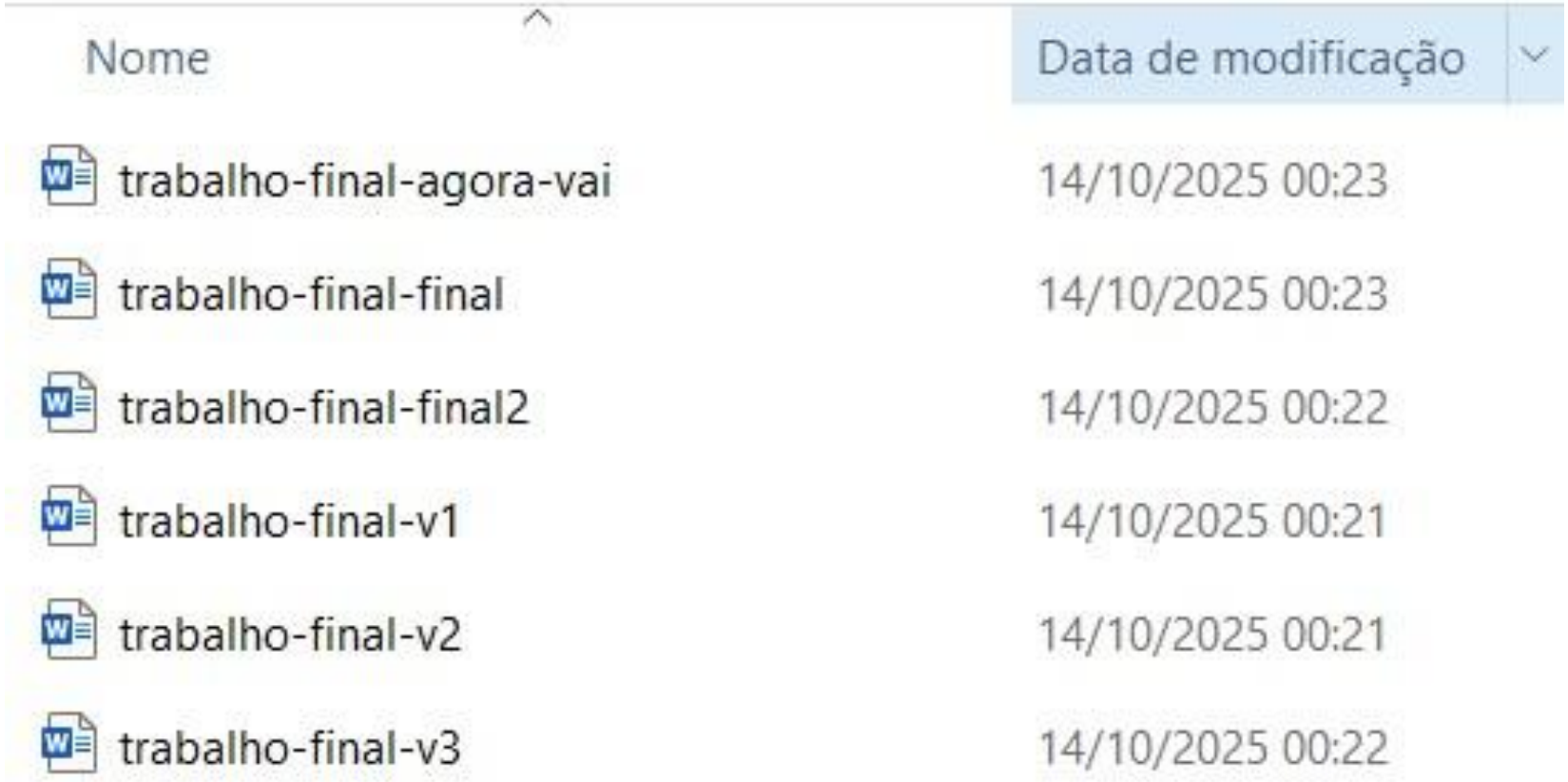
- Sistema de versionamento
- Serve para controlar versões de documentos, projetos, etc





Como assim controlar versões?

Problemática



Temos necessidade real de versionar nossos trabalhos

Manter histórico, como ele está hoje, como estava antes, etc

Então, preciso usar uma ferramenta especializada para isso...



Git





Git

É um sistema de versionamento (gerencia suas versões de trabalho) que você instala em seu computador

Ele vai permitir você manter um histórico, ver diferença entre uma versão e outra, voltar versão, trabalho em equipe, etc

Git x Github



Github

- Serviço online de hospedagem de projetos gerenciados pelo *Git*
- Você pode hospedar seus projetos de graça, você pode seguir outros usuários, outros usuários podem te seguir
- Mais de 100 milhões de desenvolvedores utilizam
- É o maior repositório de dados compartilhado do mundo
- Grandes projetos no mundo inteiro estão hospedados no *Github*

Minha página no github:

https://github.com/ericksharlls

Mas existem os concorrentes do Github

- Bitbucket - https://bitbucket.org/
- GitLab - https://gitlab.com/

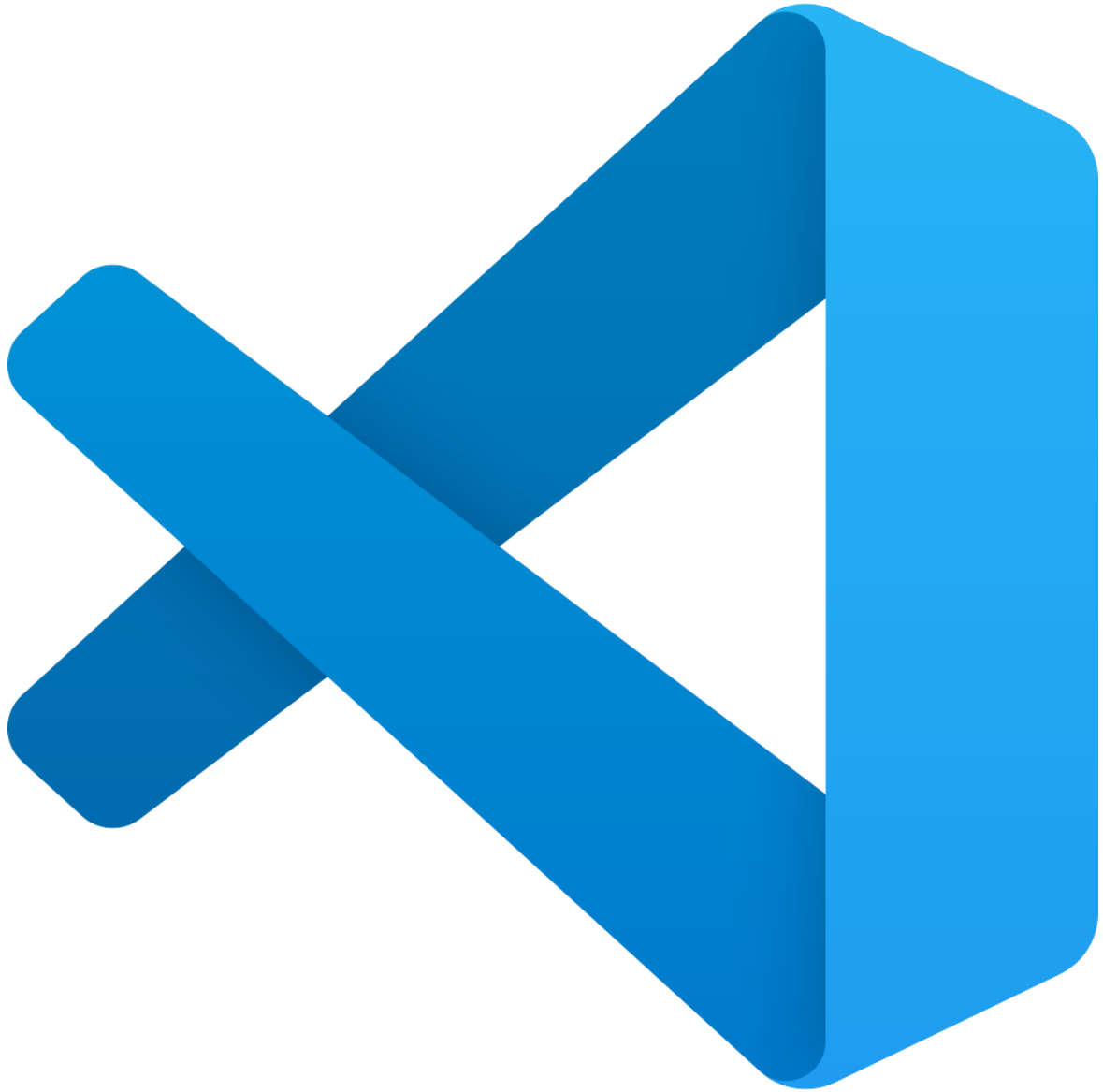
Nessa oficina usaremos...



Como verificar se o Git está instalado?

- ***git --version***
- Git instalado, agora crie uma pasta do seu projeto no computador

Vamos abrir a pasta do projeto no Visual Studio Code



Criando nosso repositório git

- Abrir o terminal do *VS Code*
- Acessar a pasta do projeto
- Criar um repositório *git* na pasta
  - ***git init***
- Agora, a pasta agora é um projeto gerenciado pelo ***git*** e um repositório de versionamento
- Foi criada uma subpasta oculta chamada *.git*

Configurando seu usuário no Git

- *git config user.name “Seu nome (não precisa ser o nome completo)”*
- *git config user.email login@dominio.com*
- *git branch -M main*

Configurando seu usuário no Git

- *git config --global user.name “Seu nome (não precisa ser o nome completo)”*
- *git config --global user.email login@dominio.com*
- *git branch -M main*
- **OBS**: use o parâmetro ***--global*** apenas no seu computador pessoal

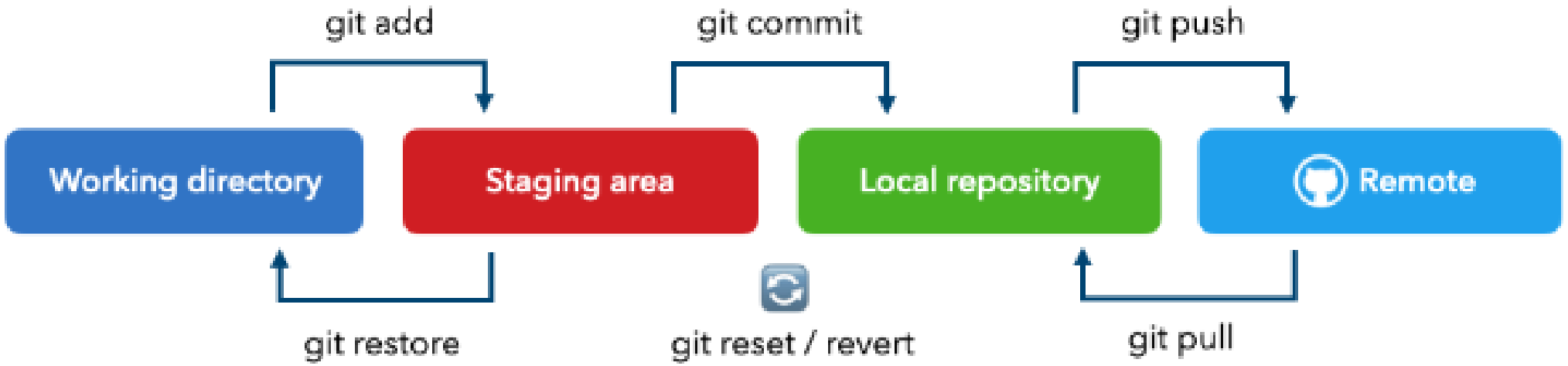
Vamos criar os arquivos *index .html* e *vendas.html*



Baixar modelos em: https://github.com/ericksharlls/arquivos-oficina-git

Ou crie arquivos *html* usando o atalho: **!** + **ENTER**

Estágios das mudanças no Git

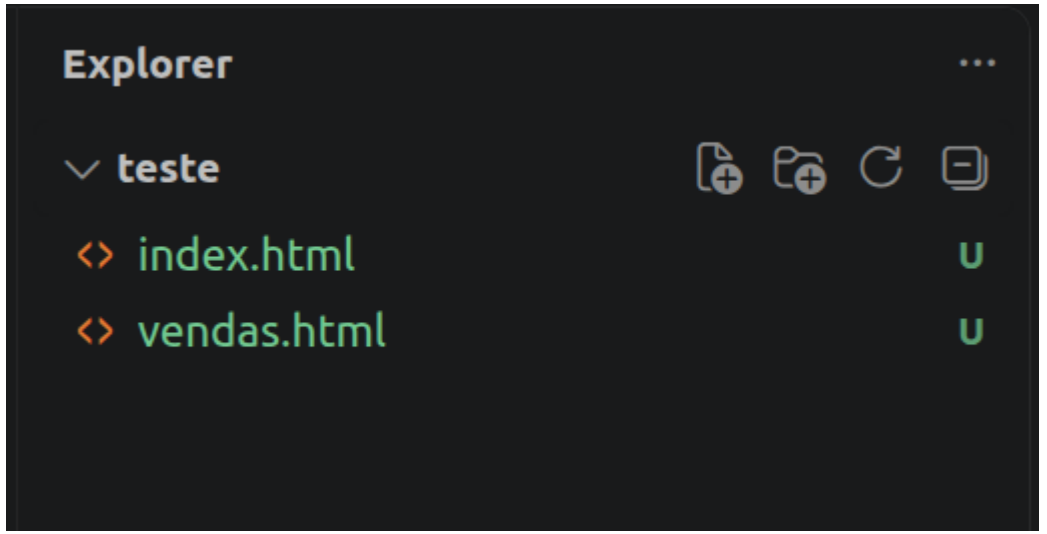


Estágios das mudanças no Git

- **Working Directory**: onde estão todos os arquivos do seu projeto
- **Staging Area**: área temporária onde você coloca os arquivos que você pretende salvar na próxima versão
  - Isso é interessante, pois você coloca os arquivos que você quiser para serem incluídos na próxima versão
- **Local Repository**: onde estão os seus arquivos confirmados (comitados) para uma nova versão

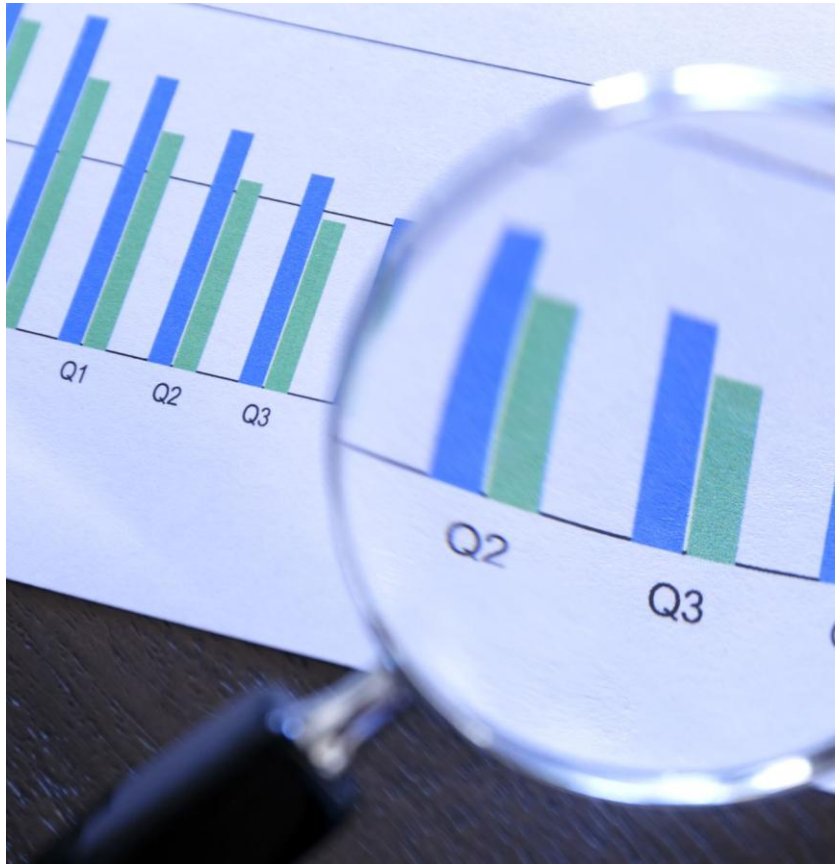
Explorando interface do VS Code

- No explorador de arquivos lateral, os arquivos ficarão na cor verde acompanhado da letra U (Untracked). Isso indica que o *Git* sabe que o arquivo existe, mas não o está "rastreando", poainda não existe versões deles



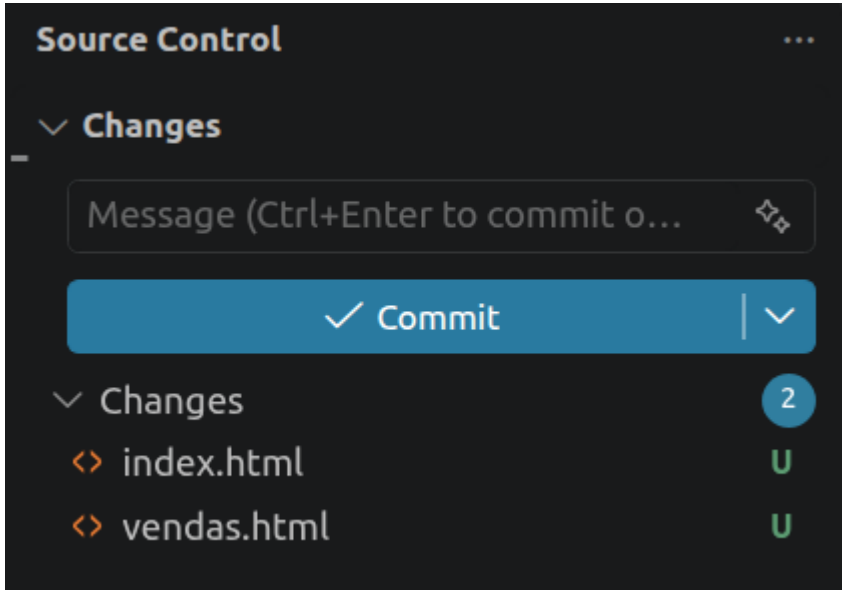
Verificando status do projeto

- Verificar em qual estágio estão as mudanças
  - *git status*



Explorando interface do VS Code

- Abra a aba ***Source Control*** (**Ctrl** + **Shift** + **G**). Observe os arquivos na seção ***Changes***.

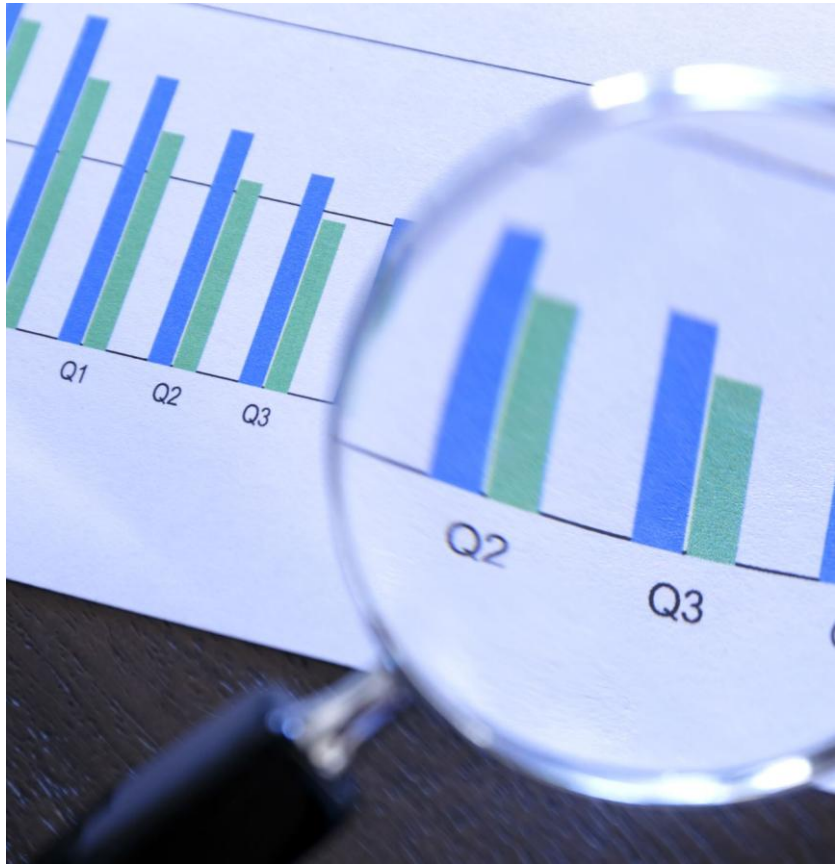


Criando a primeira versão do projeto

- Enviando para a área *staging*
  - *git add <nome_arquivo>* (para enviar um arquivo específico)
  - *git add .* (para enviar todos os arquivos)

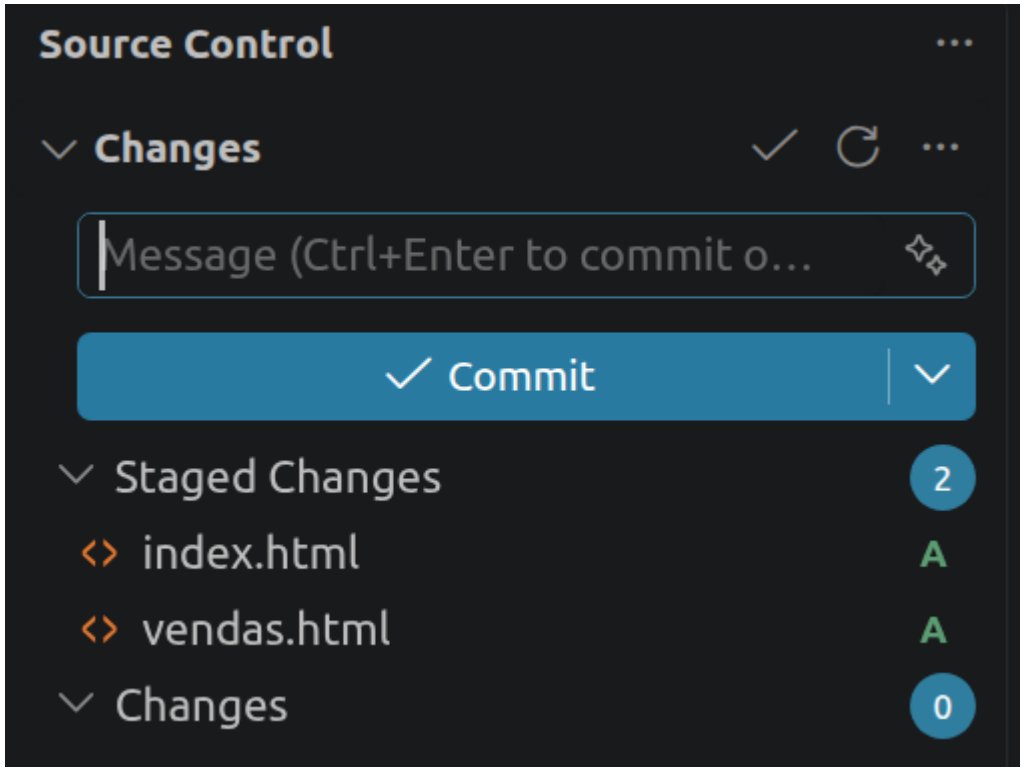
Verificando status do projeto

- Verificar em qual estágio estão as mudanças
  - *git status*



Explorando interface do VS Code

- Abra a aba de ***Source Control*** (**Ctrl + Shift + G**). O arquivo migrou da seção ***Changes*** para ***Staged Changes*** (***Staging area***) e agora exibe a letra A (Added).

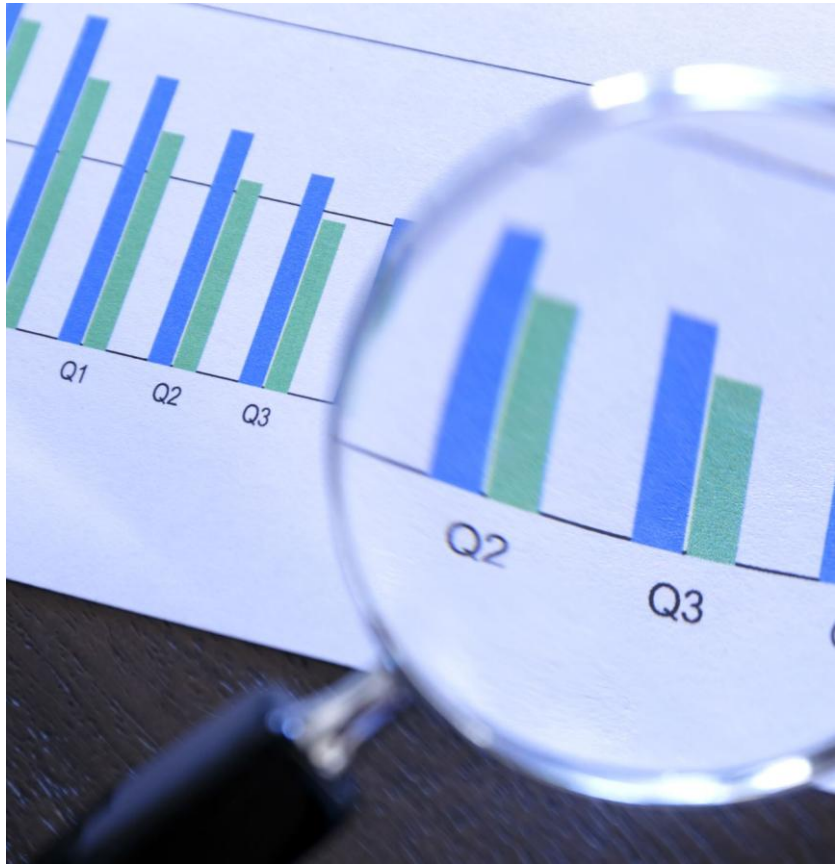


Confirmando uma nova versão (commit)

- *git commit -m “Mensagem explicativa do que está sendo feito”*

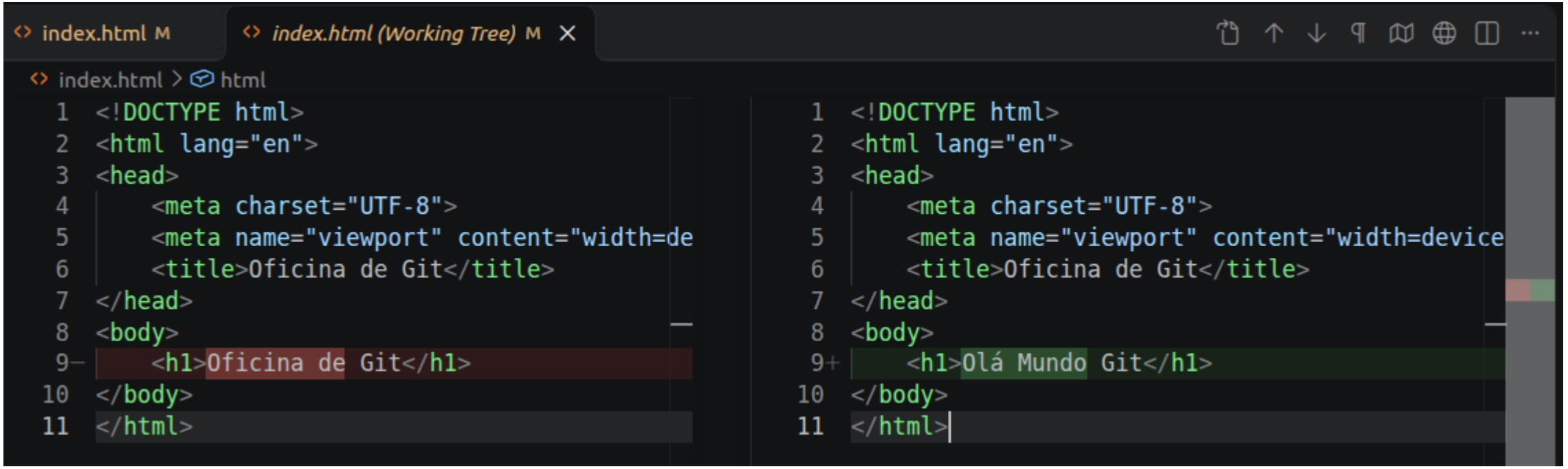
Verificando status do projeto

- Verificar em qual estágio estão as mudanças
  - *git status*

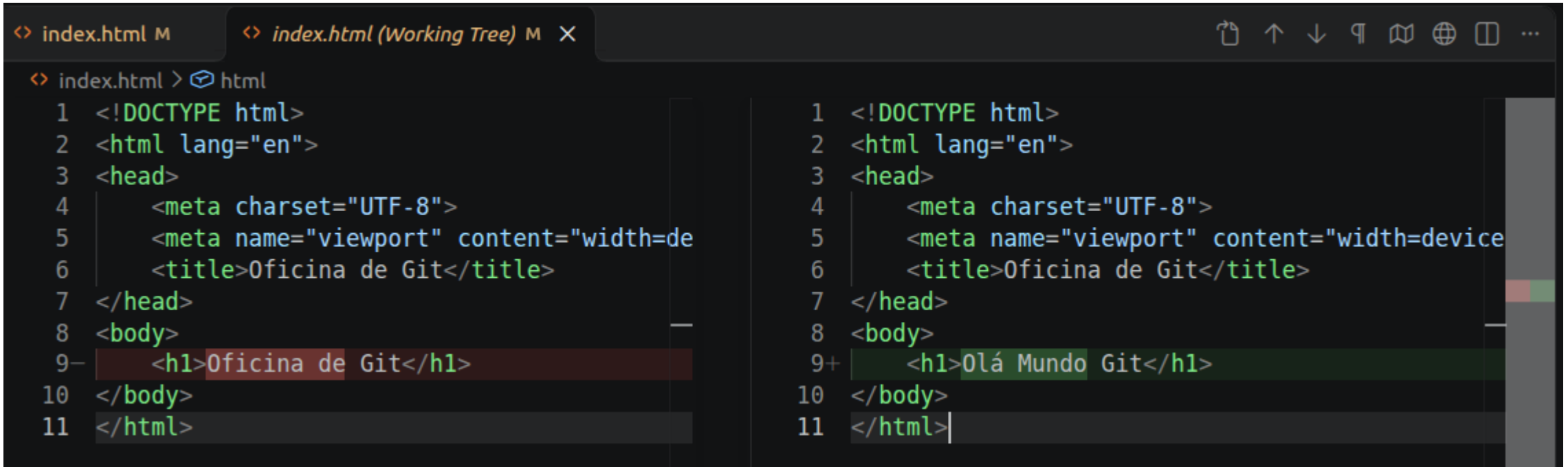


Explorando interface do VS Code

- Mude o conteúdo da página ***index.html*** e salve. Em ***Source Control***, clique em cima do arquivo ***index.html***. O VS Code abrirá uma tela dividida mostrando exatamente o que foi removido (em vermelho) e o que foi adicionado (em verde).



Explorando interface do VS Code

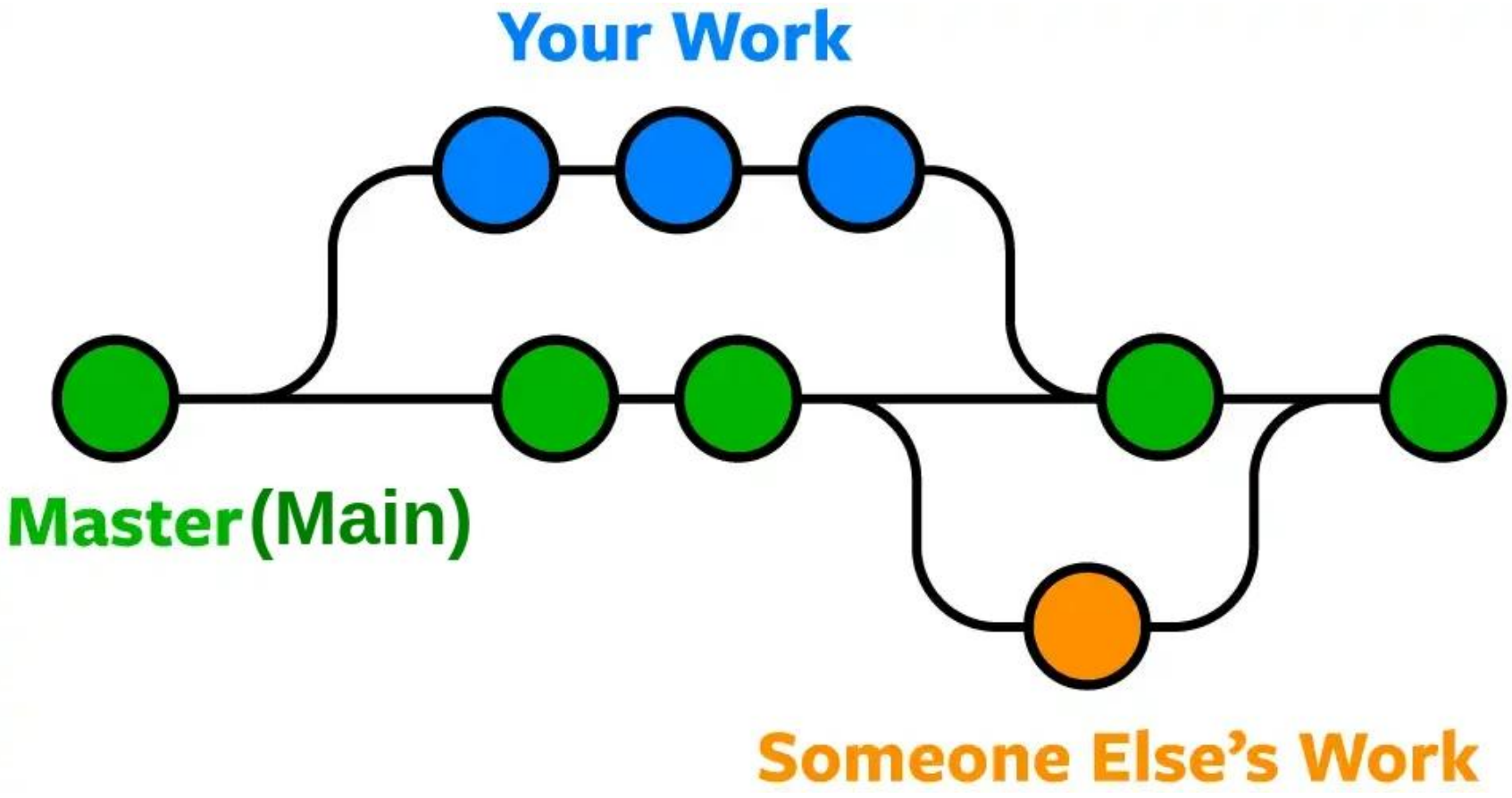


- Agora, repita o fluxo que acabamos de aprender:
  - *git add index.html*
  - *git commit -m* "Atualização da página inicial"

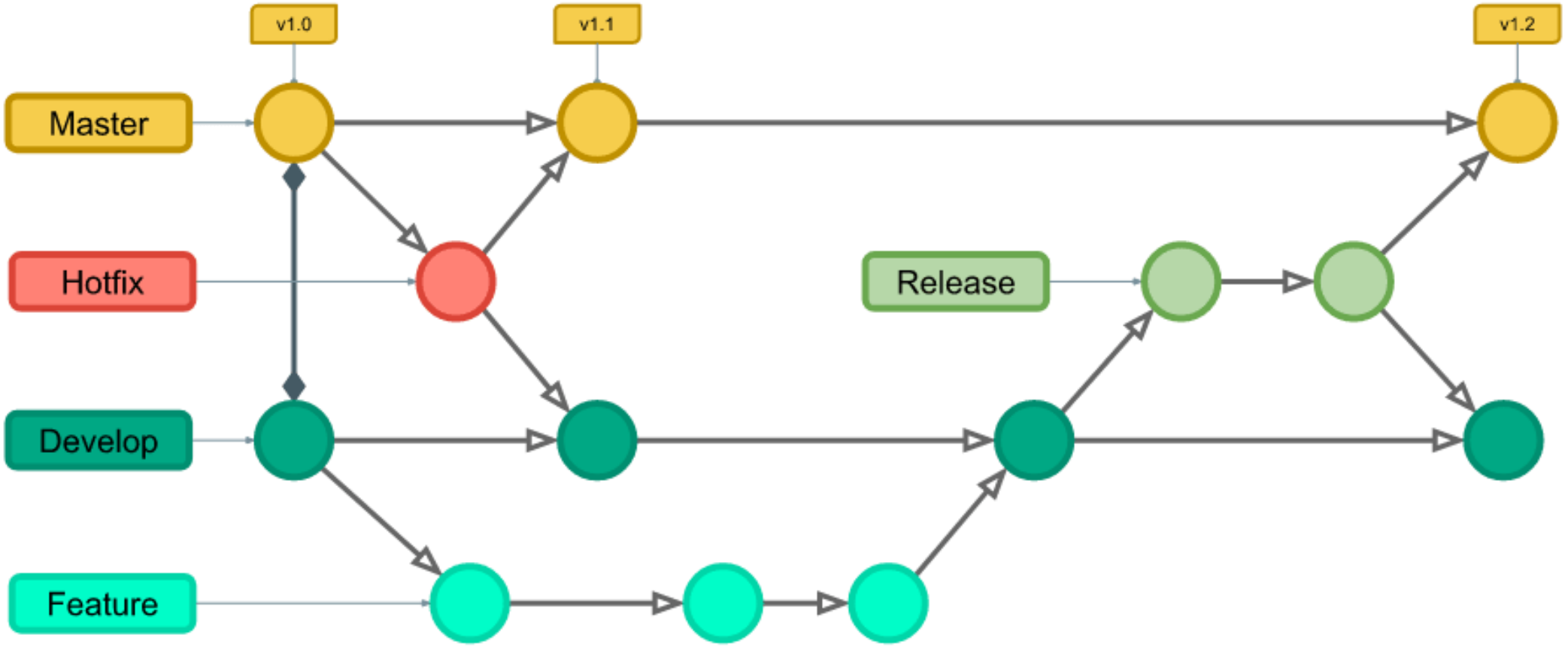
Branches

- Branches (**ramos**) são utilizados para desenvolver funcionalidades isoladas umas das outras
- O *branch* ***master/main*** é o branch padrão quando você cria um repositório
- Use outros branches para implementar novos recursos, corrigir *bugs* ou experimentar alterações no código sem afetar o projeto principal, e mescle-os (*merge*) ao *branch* ***main/master*** após a conclusão
- Branches são **caminhos** de trabalho fora do **caminho** principal

Branches



Branches - Gitflow



Criando repositório no Github

- Vamos associar o seu projeto local (no seu computador) com o projeto que está no Github
  - A associação é necessária para você enviar dados do projeto local para o remoto
- Crie um repositório **público** no Github
  - O Github sugere uma sequência de comandos para associar seu projeto local com o repositório remoto
  - Como forma de configuração, selecione HTTPS (também existe a opção SSH)
- Encontre um comando semelhante a esse:
  - *git remote add origin https://github.com/ericksharlls/oficina-git.git*

Enviando as alterações para o Github

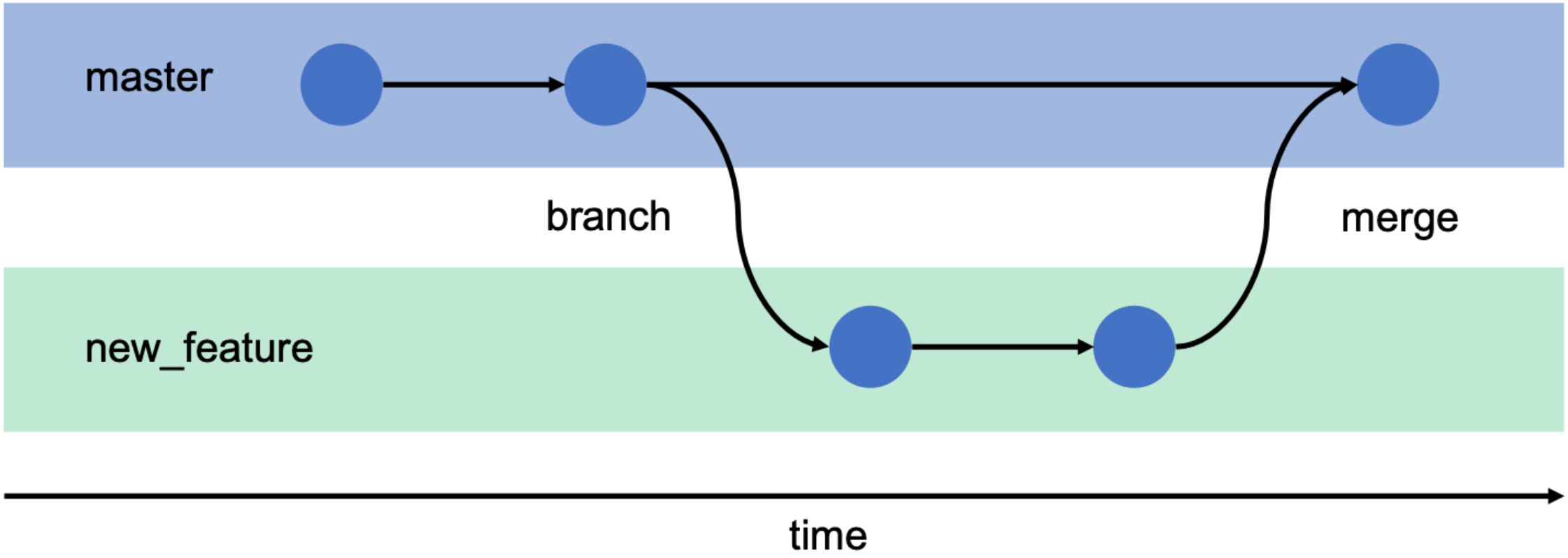
- *git push <remote> <branch>*
  - <*remote*>: Este é o nome do repositório remoto para o qual você está enviando as alterações. Por convenção, o remoto padrão é chamado de ***origin***.
  - <*branch*>: Este é o nome do *branch* local cujas alterações você deseja enviar para o repositório remoto.
- *git push origin main*
  - Este comando envia todos os commits da sua *branch* ***main*** local para a *branch* remota ***origin***

Vamos refazer o fluxo até o push para o Github?

- Altere um arquivo e depois siga o passo a passo:
  1) *git add .*
  2) *git commit -m “Mensagem explicativa do que está sendo feito”*
  3) *git push origin main*
     - *(push* envia as mudanças locais para o repositório remoto*)*

- ***git status*** a qualquer momento para verificar em qual estágio estão as mudanças
- Para descartar alterações em um arquivo no seu diretório de trabalho (*Working Directory*) e restaurá-lo ao estado em que estava no último *commit*, use:
  - *git restore <nome_arquivo>*
- Para remover um arquivo da área de *staging*, mantendo as modificações no seu diretório de trabalho, use a opção ***--staged***:
  - *git restore --staged index.html*

Trabalhando com branches

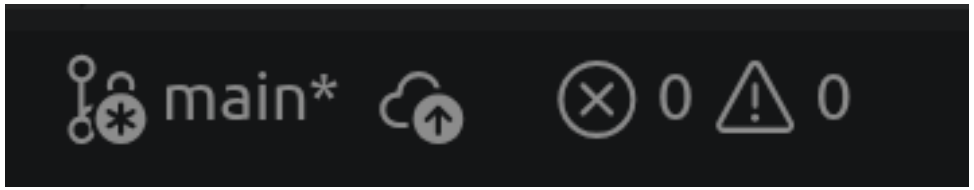


Trabalhando com branches

- Verificar em qual *branch* estou
  - *git branch*
- Selecionar a *branch* ***main***. É a partir dela que criamos outras branches
  - *git checkout main*
- Criar uma *branch* nova *(cria e seleciona a branch* ***pagina-clientes***)
  - *git checkout -b pagina-clientes*
- OBS: o ***checkout*** é utilizado para comutar/alternar entre branches

Explorando interface do VS Code

- Note que o canto inferior esquerdo do *VS Code* mostra qual ***branch*** está sendo usada no momento, ou seja, a ***branch*** do ***checkout*** mais recente. No exemplo ao lado, a ***branch*** *main* está sendo utilizada.



Trabalhando com branches

- **Passo a passo:**
  1) *git checkout -b pagina-clientes* (cria e seleciona a branch ***pagina-clientes***)
  2) Criar a página *pagina-clientes.html*
  3) *git add .*
  4) *git commit -m “Criação da página de clientes”*
  5) *git checkout main*
  6) *git merge pagina-clientes* (na branch ***main***, faça o *merge* com a branch ***pagina-clientes***)
     - ***merge*** atualiza a branch corrente com mudanças de outra branch
  7) *git push origin pagina-clientes*
  8) *git push origin main*

Atualizando seu repositório local com atualizações remotas

1) Acesse a branch ***main*** no *Github*
2) Usando o editor de código do *Github*, faça alguma alteração no arquivo ***index.html*** (clicando no ícone do lápis) e, em seguida, o *commit*
3) Na sua máquina, selecione a branch ***main*** (*git checkout main*)
4) Perceba que o arquivo ***index.html*** está desatualizado em comparação com a versão remota
5) *git pull origin main* (atualiza sua branch ***main*** local com a branch ***main*** remota)

Trabalhando com branches

Agora vamos aprender a resolver conflitos de merge

Trabalhando com branches

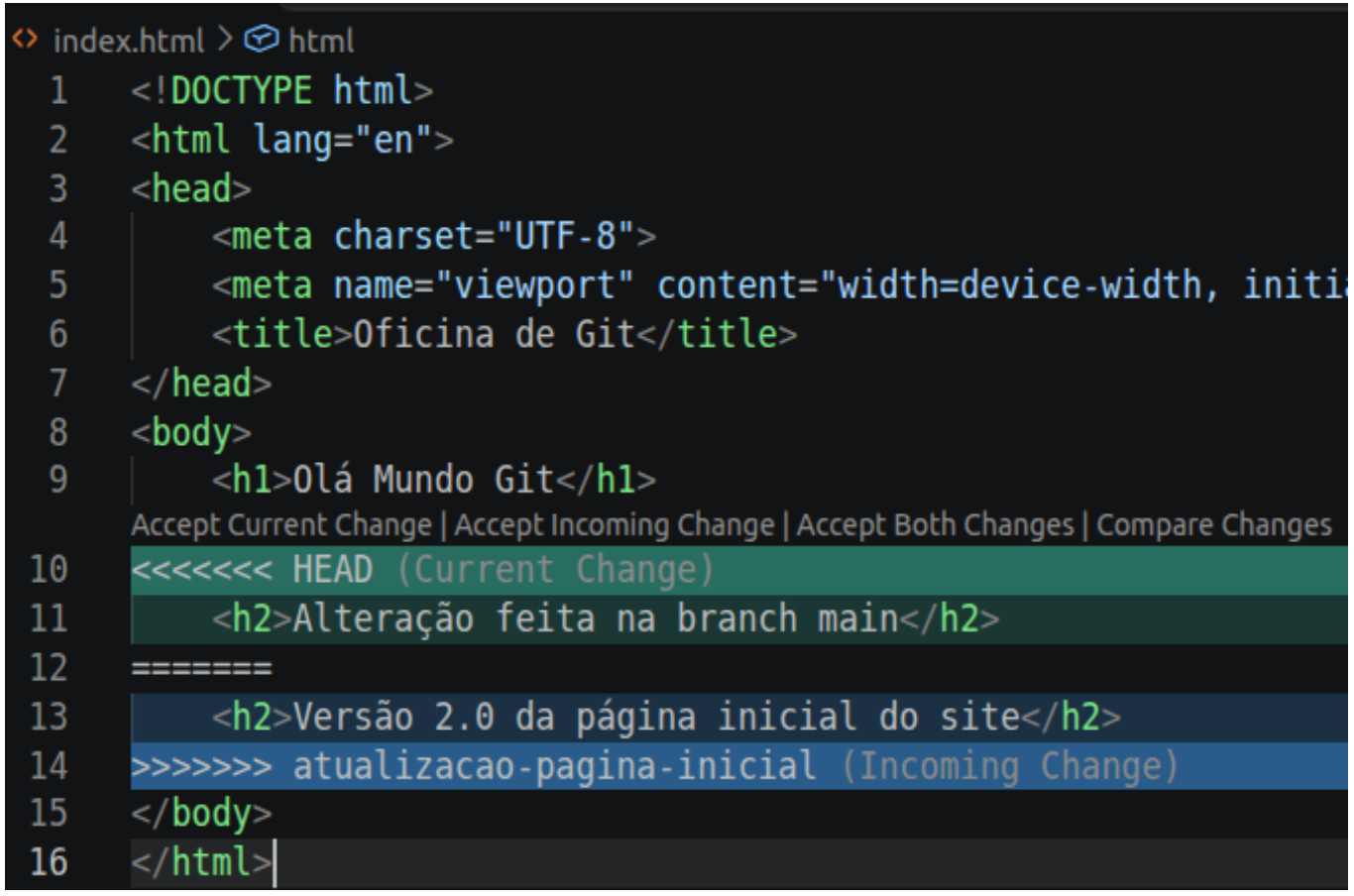
- **Simulando conflito de merge**

- **<u>Passo a passo</u>**:

1) git checkout main (seleciona a branch ***main***)
2) git checkout -b atualizacao-pagina-inicial (cria e seleciona a branch ***atualizacao-pagina-inicial***)
3) Alterar a página *index.html*
4) git add .
5) git commit -m "Nova versão da página inicial"
6) Verificar como está o arquivo *index.html* nas duas branches (***git checkout main*** e ***git checkout atualizacao-pagina-inicial***)
7) git checkout main
8) Alterar a página *index.html*
9) git add .
10) git commit -m “Atualização na branch main”

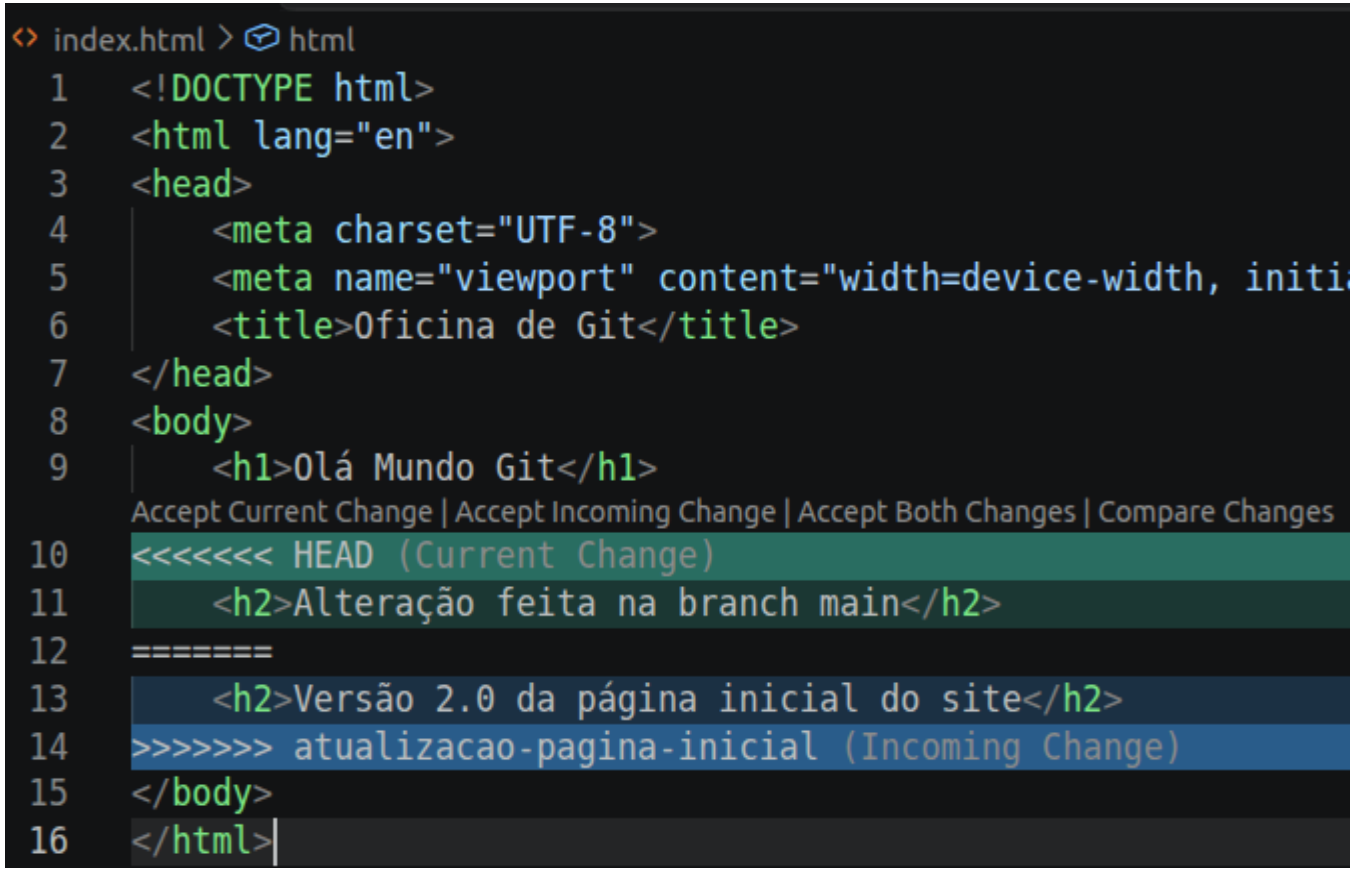
Trabalhando com branches

- **Resolvendo o conflito de merge**
- **Passo a passo:**
  11) *git merge atualizacao-pagina-inicial*
      - O terminal exibirá uma mensagem de conflito de merge. O arquivo exibirá marcações coloridas diretamente no editor.
      - Nas linhas de conflito veremos as opções:
      - *Accept Current Change*: Mantém o que está na main
      - *Accept Incoming Change*: Traz o que veio da outra branch (*atualizacao-pagina-inicial*)
      - *Accept Both Changes:* Mantém as duasalterações, uma embaixo da outra



Trabalhando com branches

- **Resolvendo o conflito de merge**
- **Passo a passo:**
  12) Entendendo os marcadores de conflito
      - *Pesquise o marcador de conflito:* **<<<<<<<**
      - *As alterações da branch base estão após a linha* **<<<<<<< *HEAD***
      - *o marcador* **=======** *divide as alterações das duas branches*
      - *após o marcador* **>>>>>>>** *está o nome da outra branch*
      - *a resolução do conflito precisa ser* ***commitada***



Trabalhando com branches

- **Resolvendo o conflito de merge**
- **Passo a passo:**
  13) *git add .*
  14) *commit -m “Resolução do conflito de merge”*
  15) *git checkout atualizacao-pagina-inicial*
  16) *git merge main (atualiza na branch **atualizacao-pagina-inicial** as últimas mudanças da branch **main**)*
  17) *git push origin atualizacao-pagina-inicial*
  18) *git push origin main*

Clonando repositório remoto no seu ambiente local

- Em qualquer computador, você consegue trabalhar em seu projeto a partir da clonagem do repositório remoto



Clonando repositório remoto no seu ambiente local

- Acessar a página do repositório no *Github*, e acessar *Code* => *Clone* => *HTTPS* (copiar a *URL*)
- *git clone <url_https>*
  - Exemplo: *git clone https://github.com/ericksharlls/arquivos-oficina-git.git*

Git não é só isso, ainda tem muito mais


Dúvidas